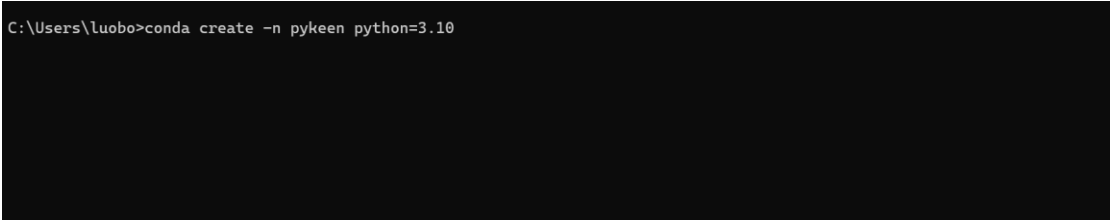


实验：使用 PyKEEN 和 Neo4j 补全知识图谱

1. 配置 conda 环境

(1) 打开命令行，创建 conda 环境

```
C:\Users\luobo>conda create -n pykeen python=3.10
```



(2) 激活 conda 环境

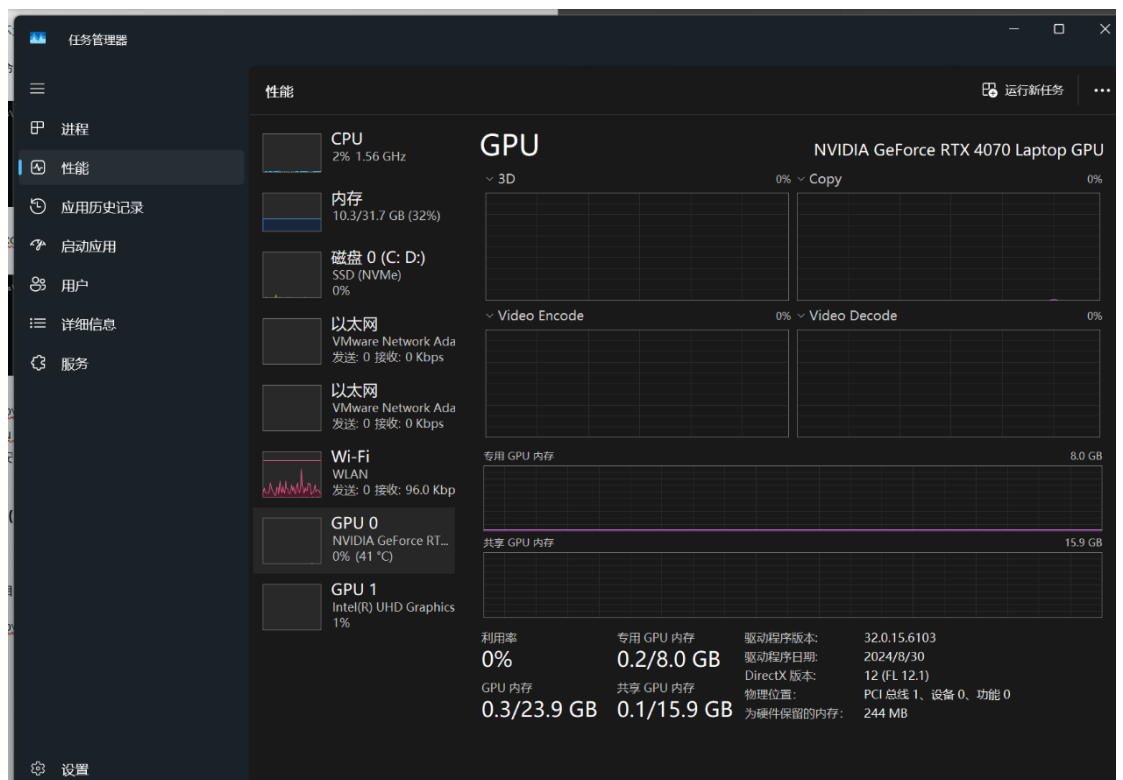
```
C:\Users\luobo>conda activate pykeen
```



(3) 安装 pytorch (推荐，若使用 cpu 版 torch 则无需此步骤，pykeen 会自动安装 cpu 版 torch，故若使用 gpu 版 torch 则需要在安装 pykeen 前或安装后重新安装 gpu 版 pytorch)

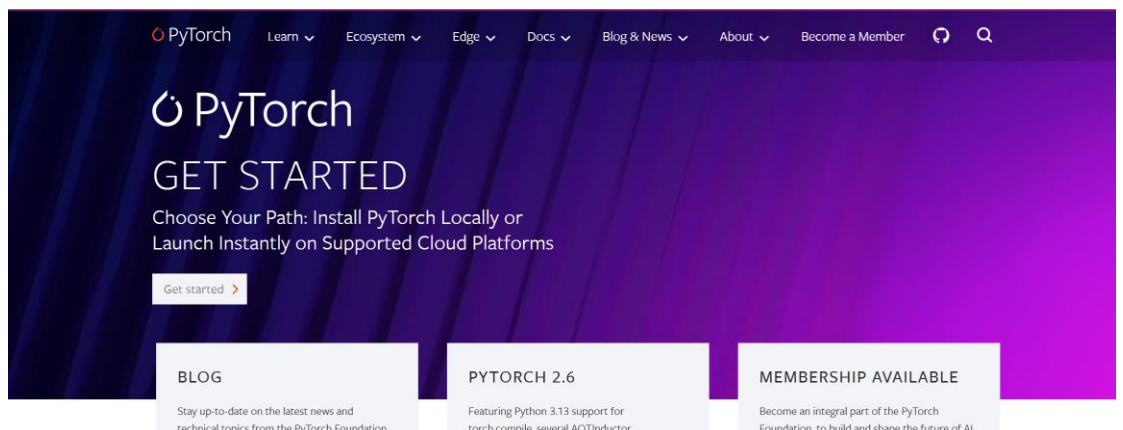
注意：请检查自己的机器是否配备 GPU 和 GPU 的显存，若没有 GPU 或总显存（专用+共享）<14G 则安装 CPU 版 pytorch，无需执行本步骤。

查看方法：在任务栏右键，打开任务管理器，切换到性能选项卡，可以看到 GPU 和显存信息。



根据自身 cuda 版本进行安装，此处以 cuda12.6 为例。

打开 pytorch 官网，点击 get started。



选择合适的版本。

PyTorch Build	Stable (2.6.0)		Preview (Nightly)	
Your OS	Linux		Mac	Windows
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.4	CUDA 12.6	ROCm 6.2.4
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126</pre>			

复制下方的命令并运行，等待安装完成。

```
(pykeen) PS C:\Users\luobo> pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126
Looking in indexes: https://download.pytorch.org/whl/cu126
Collecting torch
  Using cached https://download.pytorch.org/whl/cu126/torch-2.6.0%2Bcu126-cp310-cp310-win_amd64.whl.metadata (28 kB)
Collecting torchvision
  Using cached https://download.pytorch.org/whl/cu126/torchvision-0.21.0%2Bcu126-cp310-cp310-win_amd64.whl.metadata (6.3 kB)
Collecting torchaudio
  Using cached https://download.pytorch.org/whl/cu126/torchaudio-2.6.0%2Bcu126-cp310-cp310-win_amd64.whl.metadata (6.8 kB)
Collecting filelock (from torch)
  Using cached https://download.pytorch.org/whl/filelock-3.13.1-py3-none-any.whl.metadata (2.8 kB)
Collecting typing-extensions>=4.10.0 (from torch)
  Using cached https://download.pytorch.org/whl/typing_extensions-4.12.2-py3-none-any.whl.metadata (3.0 kB)
Collecting networkx (from torch)
  Using cached https://download.pytorch.org/whl/networkx-3.3-py3-none-any.whl.metadata (5.1 kB)
Collecting Jinja2 (from torch)
  Using cached https://download.pytorch.org/whl/Jinja2-3.1.4-py3-none-any.whl.metadata (2.6 kB)
Collecting fsspec (from torch)
  Downloading https://download.pytorch.org/whl/fsspec-2024.6.1-py3-none-any.whl.metadata (11 kB)
```

(4) 安装 pykeen 和 neo4j 驱动库。

安装最新版 pykeen（1.11 版本目前尚不支持 pytorch2.6，因此需要安装最新提交版本）。

```
(pykeen) PS C:\Users\luobo> pip install git+https://github.com/pykeen/pykeen.git
Collecting git+https://github.com/pykeen/pykeen.git
  Cloning https://github.com/pykeen/pykeen.git to c:\users\luobo\appdata\local\temp\pip-req-build-3pjxv2u7
  Running command git clone --filter=blob:none --quiet https://github.com/pykeen/pykeen.git 'C:\Users\luobo\AppData\Local\Temp\pip-req-build-3pjxv2u7'
  Resolved https://github.com/pykeen/pykeen.git to commit 6e988d52672be3d5f3a03aa084fa4aaf5db74963
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting dataclasses-json (from pykeen==1.11.1.dev0)
  Using cached dataclasses_json-0.6.7-py3-none-any.whl.metadata (25 kB)
Collecting numpy (from pykeen==1.11.1.dev0)
  Using cached numpy-2.2.4-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting scipy>=1.7.0 (from pykeen==1.11.1.dev0)
  Using cached scipy-1.15.2-cp310-cp310-win_amd64.whl.metadata (60 kB)
```

安装 neo4j 驱动库。

```
(pykeen) PS C:\Users\luobo> pip install neo4j -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting neo4j
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/6a/57/94225fe5e9dabdc0ff60c88cbfcedf11277f4b34e7ab1373d3e62dbdd207/neo4j-5.28.1-py3-none-any.whl (312 kB)
Requirement already satisfied: pytz in d:\envs\scoop\persist\anaconda3\app\envs\pykeen\lib\site-packages (from neo4j) (2025.2)
Installing collected packages: neo4j
Successfully installed neo4j-5.28.1
```

(5) 将 conda 配置到 notebook

```
(pykeen) PS C:\Users\luobo> pip install ipykernel
Collecting ipykernel
  Using cached ipykernel-6.29.5-py3-none-any.whl.metadata (6.3 kB)
Collecting comm>=0.1.1 (from ipykernel)
  Using cached comm-0.2.2-py3-none-any.whl.metadata (3.7 kB)
Collecting debugpy>=1.6.5 (from ipykernel)
  Downloading debugpy-1.8.13-cp310-cp310-win_amd64.whl.metadata (1.3 kB)
Collecting ipython>=7.23.1 (from ipykernel)
  Downloading ipython-8.35.0-py3-none-any.whl.metadata (5.1 kB)
Collecting jupyter-client>=6.1.12 (from ipykernel)
```

把环境作为内核注册到 notebook

```
(pykeen) C:\Users\luobo>python -m ipykernel install --user --name=pykeen
Installed kernelspec pykeen in C:\Users\luobo\AppData\Roaming\jupyter\kernels\pykeen
```

(6) 检查配置是否成功，出现 pykeen 即为成功。

```
PS C:\Users\luobo> conda activate pykeen
(pykeen) PS C:\Users\luobo> jupyter kernelspec list
Available kernels:
  python3      D:\envs\scoop\apps\anaconda3\current\AppData\Local\Envs\pykeen\share\jupyter\kernels\python3
  pykeen       C:\Users\luobo\AppData\Roaming\jupyter\kernels\pykeen
(pykeen) PS C:\Users\luobo>
```

2. 启动 neo4j

(1) 使用实验材料中的 neo4j 目录（已配置好环境并已导入好数据），启动 neo4j。

```
about the exact format. This variable is incompatible with new_size and takes precedence over new_size.
PS D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5> .\bin\neo4j.bat console
Directories in use:
home:          D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5
config:        D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\conf
logs:          D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\logs
plugins:       D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\plugins
import:        D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\import
data:          D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\data
certificates:  D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\certificates
licenses:      D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\licenses
run:           D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\run
Starting Neo4j.
WARNING! You are using an unsupported Java runtime.
* Please use Java(TM) 17 or Java(TM) 21 to run Neo4j.
* Please see https://neo4j.com/docs/ for Neo4j installation instructions.
2025-04-08 16:08:30.517+0000 INFO  Logging config in use: File 'D:\progress\charset-shelter\pykeen\neo4j-community-5.26.5\conf\user-logs.xml'
2025-04-08 16:08:30.525+0000 INFO  Starting...
2025-04-08 16:08:31.662+0000 INFO  This instance is ServerId{1f7c0eb3} (1f7c0eb3-0c6a-4593-b3f3-4a1cf2407469)
2025-04-08 16:08:32.720+0000 INFO  ===== Neo4j 5.26.5 =====
2025-04-08 16:08:34.514+0000 INFO  Anonymous Usage Data is being sent to Neo4j, see https://neo4j.com/docs/usage-data/
2025-04-08 16:08:34.658+0000 INFO  Bolt enabled on localhost:7687.
2025-04-08 16:08:35.575+0000 INFO  HTTP enabled on localhost:7474.
2025-04-08 16:08:35.575+0000 INFO  Remote interface available at http://localhost:7474/
2025-04-08 16:08:35.575+0000 INFO  id: A7F38D0EF3FE2A8579E7BE1D252B2F55ED711B62588E94F5280E2A9E13C5700A
2025-04-08 16:08:35.575+0000 INFO  name: system
2025-04-08 16:08:35.575+0000 INFO  creationDate: 2025-04-08T13:33:10.102Z
2025-04-08 16:08:35.575+0000 INFO  Started.
```

看到"Started"证明 neo4j 启动成功。

如果想使用已有的 neo4j，则需要进行数据导入，执行下列步骤

为兼容性起见，建议使用 neo4j 5.26.5 版本。

导入 dump 文件：

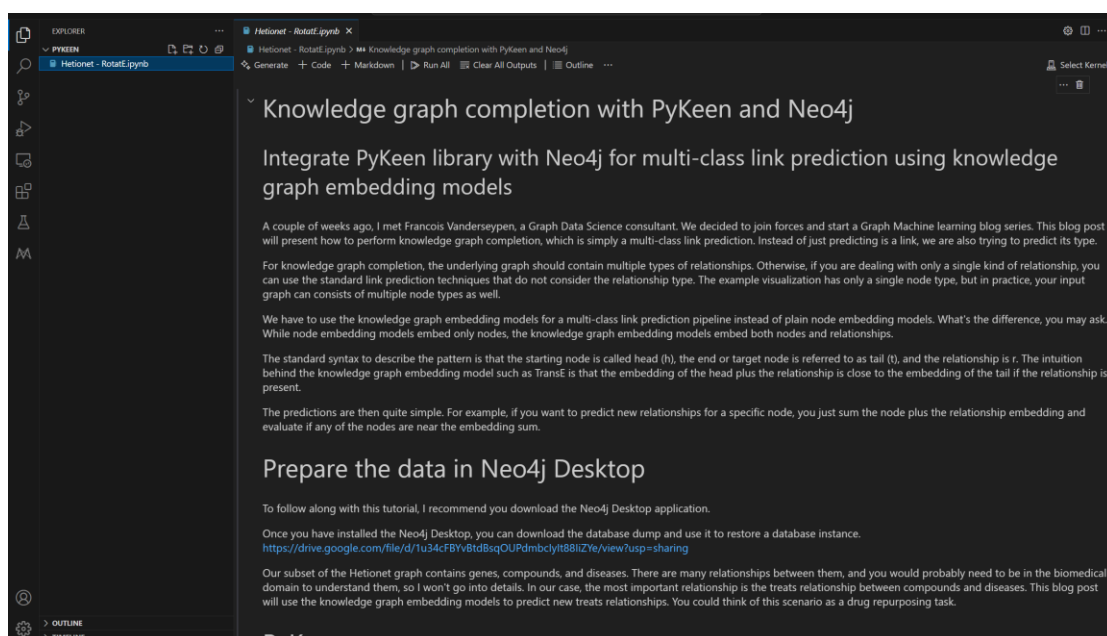
若右键菜单中无此选项，则可以打开一个新的命令行，进入所在目录。

```
Loading personal and system profiles took 2110ms.
PS C:\Users\luobo> d:
PS D:\> cd D:\progress\charset-shelter\pykeen
PS D:\progress\charset-shelter\pykeen> |
```

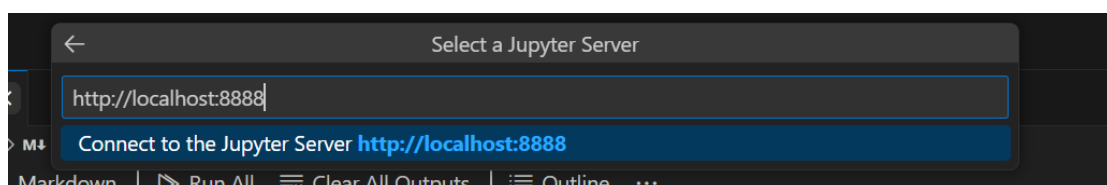
记录 token（地址中 token=之后的部分，如本案例中 token 前 6 位为 50d50a）

```
Or copy and paste one of these URLs:
http://localhost:8888/lab?token=50d50a4afb204445d4d0073aa4770251b8aede9ac697fa5a
http://127.0.0.1:8888/lab?token=50d50a4afb204445d4d0073aa4770251b8aede9ac697fa5a
```

(2) 在 vscode 中打开 hetionet-rotatE.ipynb 文件。

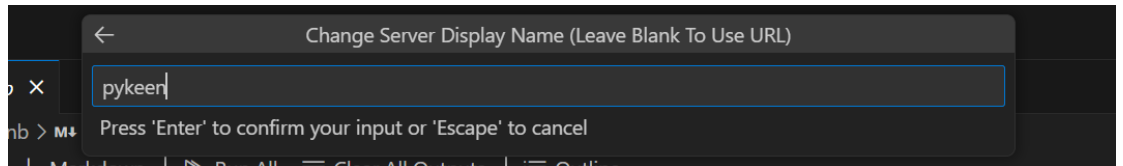


(3) 在右上角点击 select kernel -> Existing Jupyter Server，输入 server 地址。

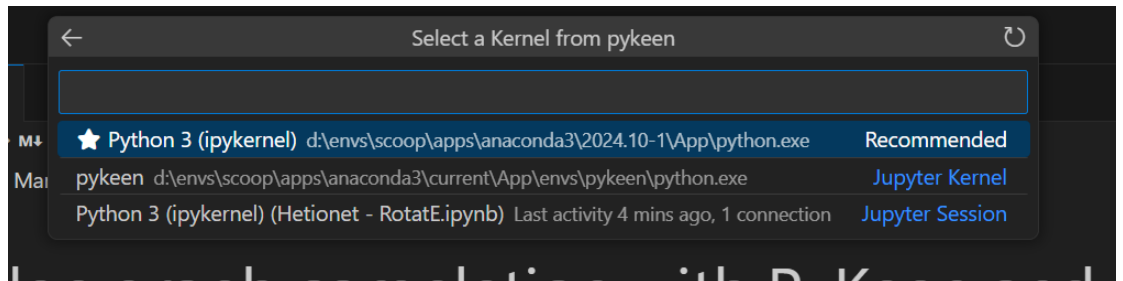


(4) 将（1）中记录的 token 作为 password 输入。

(5) 起名为 pykeen。



- (6) 选择 pykeen。



4. 实验原理与步骤

- (1) 实验目的

本实验将利用 Hetionet 数据集，实现图谱的链接预测。

- (2) 数据集介绍

Hetionet 数据集是一个生物信息学的知识图谱，总计包括 47031 个节点和 2250197 条边，本实验使用其中的子集。

- (3) 构建三元组

Pykeen 可以根据图谱的三元组信息，构建出知识图谱的内部表示。



- (4) 训练知识图谱嵌入模型

Pykeen 提供了 pipeline 方法，用于快捷训练知识图谱嵌入模型，本次实验选择使用 RotatE 算法。

```

from pykeen.pipeline import pipeline

result = pipeline(
    training=training,
    testing=testing,
    validation=validation,
    model='RotatE',
    stopper='early',
    epochs=20,
    dimensions=512,
    random_seed=420
)

```

(5) 执行多路链接预测。

```

from pykeen import predict

compound_id = run_query("""
MATCH (s:Compound)
WHERE s.name = "L-Asparagine"
RETURN toString(id(s)) as id
""")['id'][0]

df = predict.predict_target(model=result.model, head=compound_id, relation="treats", triples_factory=result.training).df
print(df.head(5))

```

(variable) compound_id: Any

WARNING:neo4j.notifications:Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.FeatureDeprecationWarning} {category: ...}

tail_id	score	tail_label
3784	-6.708349	13889
18145	-6.711460	8189
3959	-6.725380	14065
6946	-6.754873	1712
968	-6.758554	110

(6) 存储回 neo4j。

(7) 查看预测结果

Inspect results

Generate + Code + Markdown

```

run_query("""
MATCH (c:Compound)-[:PREDICTED_TREATS]->(d:Disease)
RETURN c.name as compound, d.name as disease
""")

```

	compound	disease
0	L-Asparagine	schizophrenia
1	L-Asparagine	bipolar disorder
2	L-Asparagine	type 2 diabetes mellitus

5. 执行 notebook 代码

【实验任务】完成 notebook 文件 pykeen-neo4j.ipynb 中的内容：

将 notebook 每个代码段执行成功的输出截图粘贴在下面

【代码段 1 执行截图】：

```
def run_query(query, params={}):
    with driver.session() as session:
        result = session.run(query, params)
        return pd.DataFrame([r.values() for r in result], columns=result.keys())
```

0.4s Python

【代码段 2 执行截图】：

```
Received notification from DBMS server: <GqlStatusObject gql_status='01N02', status_description='warn: feature deprecated
Received notification from DBMS server: <GqlStatusObject gql_status='01N02', status_description='warn: feature deprecated
```

0.6s Python

【代码段 3 执行截图】：

Name	Type	Size	Value
data	DataFrame	(32041, 3)	source target type
driver	BoltDriver		<neo4j_sync.driver.BoltDriver object at 0x0000...
host	str	21	'bolt://localhost:7687'
password	str	15	's3024244236.com'
TARGET_COMPO	str	12	'L-Asparagine'
user	str	5	'neo4j'

【代码段 4 执行截图】：

Name	Type	Size	Value
data	DataFrame	(32041, 3)	source target type
driver	BoltDriver		<neo4j_sync.driver.BoltDriver object at 0x0000...
host	str	21	'bolt://localhost:7687'
password	str	15	's3024244236.com'
TARGET_COMPO	str	12	'L-Asparagine'
tf	TriplesFactory		TriplesFactory(num_entities=1700, num_relatic...
user	str	5	'neo4j'

【代码段 5 执行截图】：

Name	Type	Size	Value
data	DataFrame	(32041, 3)	source target type
driver	BoltDriver		<neo4j_sync.driver.BoltDriver object at 0x0000...
host	str	21	'bolt://localhost:7687'
password	str	15	's3024244236.com'
TARGET_COMPO	str	12	'L-Asparagine'
testing	TriplesFactory		TriplesFactory(num_entities=1700, num_relati...
tf	TriplesFactory		TriplesFactory(num_entities=1700, num_relati...
training	TriplesFactory		TriplesFactory(num_entities=1700, num_relati...
user	str	5	'neo4j'
validation	TriplesFactory		TriplesFactory(num_entities=1700, num_relati...

【代码段 6 执行截图】：

```
*** No cuda devices were available. The model runs on CPU
D:\Tool\program\Python\Python311\Lib\site-packages\torch\utils\data\data_loader.py:775: UserWarning: 'pin_memory' argument
super().__init__(loader)

*** Training epochs on cpu: 100% 5/5 [00:02<00:00, 1.77epoch/s, loss=0.636, prev_loss=0.699]

*** WARNING:pykeen.utils:Using automatic batch size on device.type='cpu' can cause unexplained out-of-memory crashes. Therefor

*** Evaluating on cpu: 100% 3.20k/3.20k [00:02<00:00, 1.48ktriple/s]

*** WARNING:torch_max_mem.api:Encountered tensors on device_types={'cpu'} while only ['cuda'] are considered safe for automati
INFO:pykeen.evaluation.evaluator:Evaluation took 2.49s seconds
```

【代码段 7 执行截图】：

```
INFO:pykeen.triples.triples_factory:Stored TriplesFactory(num_entities=1700, num_relations=10, create_inverse_triples=False)
INFO:pykeen.pipeline.api:Saved to directory: C:\Users\TH\model
```

【代码段 8 执行截图】：

```
WARNING:neo4j.notifications:Received notification from DBMS server: <GqlStatusObject gql_status='01N02', status_descriptio
tail_id score tail_label
1197 1197 -6.106464 40
60 60 -8.012221 10743
785 785 -8.037053 19467
997 997 -8.058396 22093
764 764 -8.096894 19193
```

【代码段 9 执行截图】：

```
WARNING:neo4j.notifications:Received notification from DBMS server: <GqlStatusObject gql_status='01N02', status_descriptio
WARNING:neo4j.notifications:Received notification from DBMS server: <GqlStatusObject gql_status='01N02', status_descriptio
```

testing	TriplesFactory	TriplesFactory(num_entities=1700, num_relati
tf	TriplesFactory	TriplesFactory(num_entities=1700, num_relati
training	TriplesFactory	TriplesFactory(num_entities=1700, num_relati
user	str	5 'neo4j'
validation	TriplesFactory	TriplesFactory(num_entities=1700, num_relati

【代码段 10 执行截图】：

	compound	disease
0	L-Asparagine	schizophrenia
1	L-Asparagine	bipolar disorder
2	L-Asparagine	type 2 diabetes mellitus
3	L-Asparagine	obesity

【代码段 11 执行截图】：

[n in nodes(p) | n.name]

0	[L-Asparagine, ASRGL1, GDF15, colon cancer]
1	[L-Asparagine, SLC1A5, VEGFA, colon cancer]
2	[L-Asparagine, SLC38A3, VEGFA, colon cancer]
3	[L-Asparagine, NARS, TP53, colon cancer]
4	[L-Asparagine, ASNS, CCNB1, colon cancer]
5	[L-Asparagine, SLC1A5, FZD5, colon cancer]
6	[L-Asparagine, ASRGL1, AKR7A2, colon cancer]
7	[L-Asparagine, SLC38A3, MIF, colon cancer]
8	[L-Asparagine, SLC1A5, GNAI1, colon cancer]
9	[L-Asparagine, ASRGL1, PUF60, colon cancer]
10	[L-Asparagine, ASNS, CEBPB, colon cancer]
11	[L-Asparagine, ASNS, HSF1, colon cancer]
12	[L-Asparagine, SLC1A5, RUVBL1, colon cancer]
13	[L-Asparagine, ASRGL1, PSMG1, colon cancer]
14	[L-Asparagine, SLC1A5, G3BP1, colon cancer]
15	[L-Asparagine, ASNS, G3BP1, colon cancer]
16	[L-Asparagine, ASRGL1, CDK7, colon cancer]
17	[L-Asparagine, ASNS, BRAF, colon cancer]
18	[L-Asparagine, SLC38A3, MT1X, colon cancer]
19	[L-Asparagine, SLC1A5, OXCT1, colon cancer]
20	[L-Asparagine, SLC38A3, EZH2, colon cancer]
21	[L-Asparagine, ASRGL1, NME1, colon cancer]
22	[L-Asparagine, SLC38A3, RBM28, colon cancer]
23	[L-Asparagine, SLC38A3, PLXNA1, colon cancer]
24	[L-Asparagine, SLC1A5, APC, colon cancer]